

IT Service Desk Resolution Copilot — Solution Architecture

Item	Detail
Document type	Solution Architecture Document
Use case	IT Service Desk Resolution Copilot (Similar Past Incidents with AI Insight)
System of record	ServiceNow
Tech stack	Node.js + TypeScript (backend), React.js (frontend), MongoDB Atlas Vector Search, Groq / OpenAI (LLM), OpenAI embeddings, AWS
Status	Baseline v1.0 — derived strictly from the 20-question architecture questionnaire

1. Problem Statement

Service Desk agents (L1/L2) spend a large share of their handling time searching old tickets and runbooks for fixes to problems that have already been solved. Knowledge is spread across thousands of closed ServiceNow records and Confluence pages, keyword search misses tickets described in different words, and fixes for recurring issues are rediscovered again and again. The result is a high Mean Time To Resolve (MTTR) and inconsistent resolution quality.

Solution: when an Incident is created in ServiceNow, the Copilot automatically finds the most relevant resolved Incidents, Problems and Changes, summarises each one (issue, root cause, resolution), and produces one consolidated insight grounded in the organisation's runbooks. The result is posted to the ticket as a Work Note, with a deep link to a React application for full details, on-demand refresh and feedback.

1.1 Prompt Framework (ICE POT) used for this design

Element	Content
Instruction	Design the architecture for an IT Service Desk Resolution Copilot that retrieves similar past incidents and generates grounded summaries and insights.
Context	Inputs: ServiceNow Incident/Problem/Change records (V) and Confluence runbooks/SOPs (V). Critical pipelines: ingestion and retrieval.
Examples	Ticket " <i>Unable to connect to VPN, error 809</i> " → similar resolved incidents + linked Problem + runbook-grounded insight.
Persona	Chief AI Architect, 10+ years of experience.
Output	Architecture document (<code>.md</code>) with API endpoints for each functionality.
Tone	Professional and formal.

2. Scope

2.1 In scope

Area	Scope
Input sources	ServiceNow Incident, Problem and Change records (V); Confluence runbooks / SOPs / technical documentation (V)
Output	Ranked similar past incidents with an LLM summary per incident, plus a consolidated LLM insight
Delivery	ServiceNow Work Note (concise, audit trail) + standalone React app (full detail)
Trigger	Automatic on Incident creation + on-demand refresh
Critical pipelines	Ingestion pipeline; Retrieval pipeline

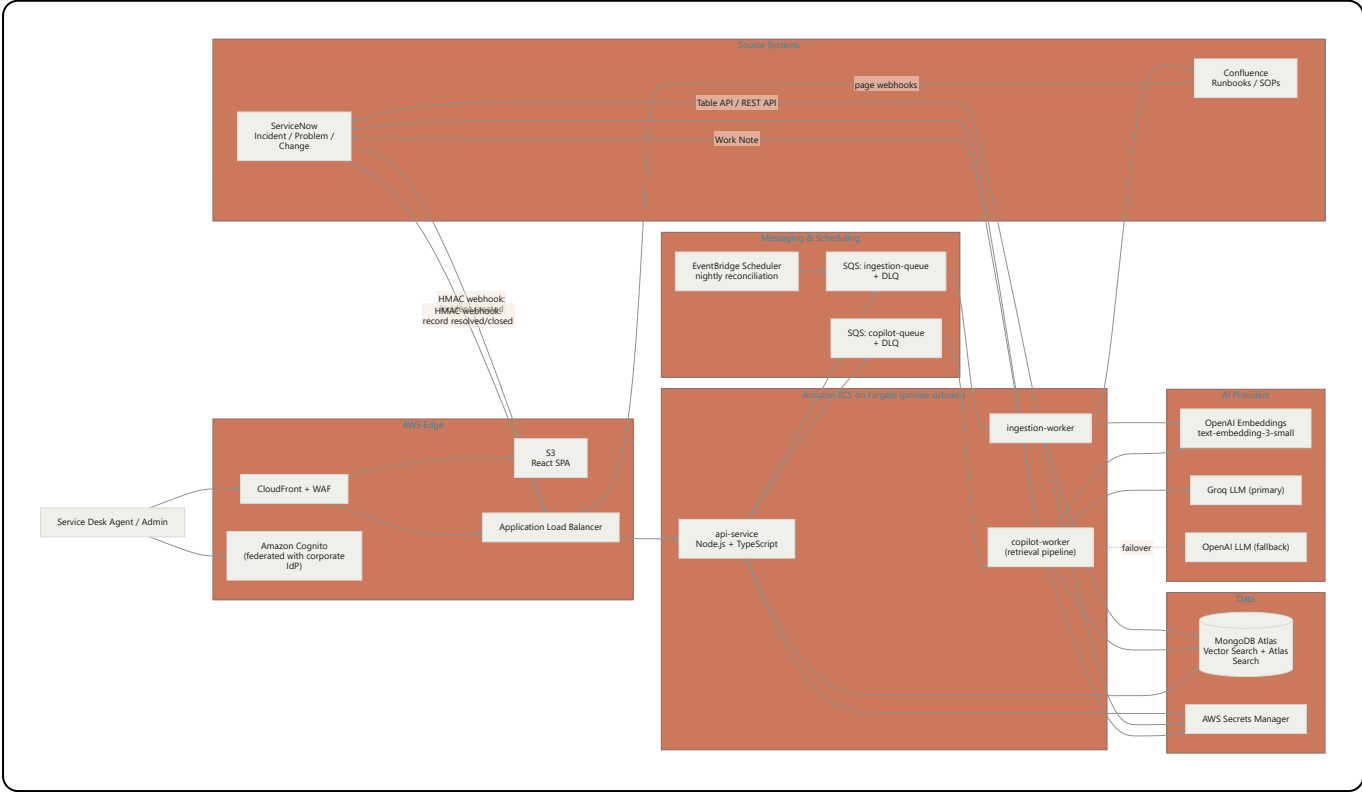
2.2 Out of scope (explicitly excluded by questionnaire decisions)

Excluded item	Decision reference
ServiceNow Knowledge Base articles	Q3
Teams / Slack threads and post-mortems	Q3
CMDB and monitoring alerts	Q3
Auto-triage (category, priority, assignment group)	Q4
Generated resolution steps as separate output	Q4
Draft customer replies	Q4
React UI embedded inside ServiceNow forms	Q7
LLM-based query rewriting	Q12
Dedicated cross-encoder reranking model	Q14

3. Architecture Decision Log

#	Topic	Decision
Q1	Use case	IT Service Desk Resolution Copilot
Q2	ITSM platform	ServiceNow
Q3	Additional sources	Runbooks / SOPs / Confluence (V)
Q4	Output	Similar past incidents
Q5	Presentation	Ranked list + per-incident LLM summary + consolidated LLM insight
Q6	Trigger	Automatic on creation + on-demand refresh
Q7	Delivery channel	ServiceNow Work Note + standalone React app
Q8	Sync strategy	Initial backfill + event-driven sync + nightly reconciliation
Q9	Record qualification	Resolved/Closed with resolution notes + 24-month window + quality filter
Q10	Chunking	Structure-aware (ticket = 1 chunk) + parent-child for runbooks
Q11	Embedding model	OpenAI (text-embedding-3-small) (1536 dims) behind a provider abstraction
Q12	Query preprocessing	Curated, admin-managed dictionary
Q13	Hybrid search	Vector + BM25, Reciprocal Rank Fusion, metadata pre-filters
Q14	Reranking	Groq LLM relevance scoring + heuristic business rules
Q15	Deduplication	Exact ID + parent-child collapse + near-duplicate clustering + Problem/Parent grouping
Q16	LLM strategy	Groq primary, OpenAI automatic fallback
Q17	Data protection	PII masking at ingestion + secrets detection + query-time masking
Q18	Auth	Corporate SSO (Amazon Cognito) + RBAC (Agent/Admin) + HMAC-signed webhooks
Q19	Compute	Amazon ECS on Fargate + SQS + EventBridge Scheduler
Q20	Quality & observability	Monitoring + agent feedback + golden-set evaluation + KPI & cost dashboard

4. High-Level Architecture



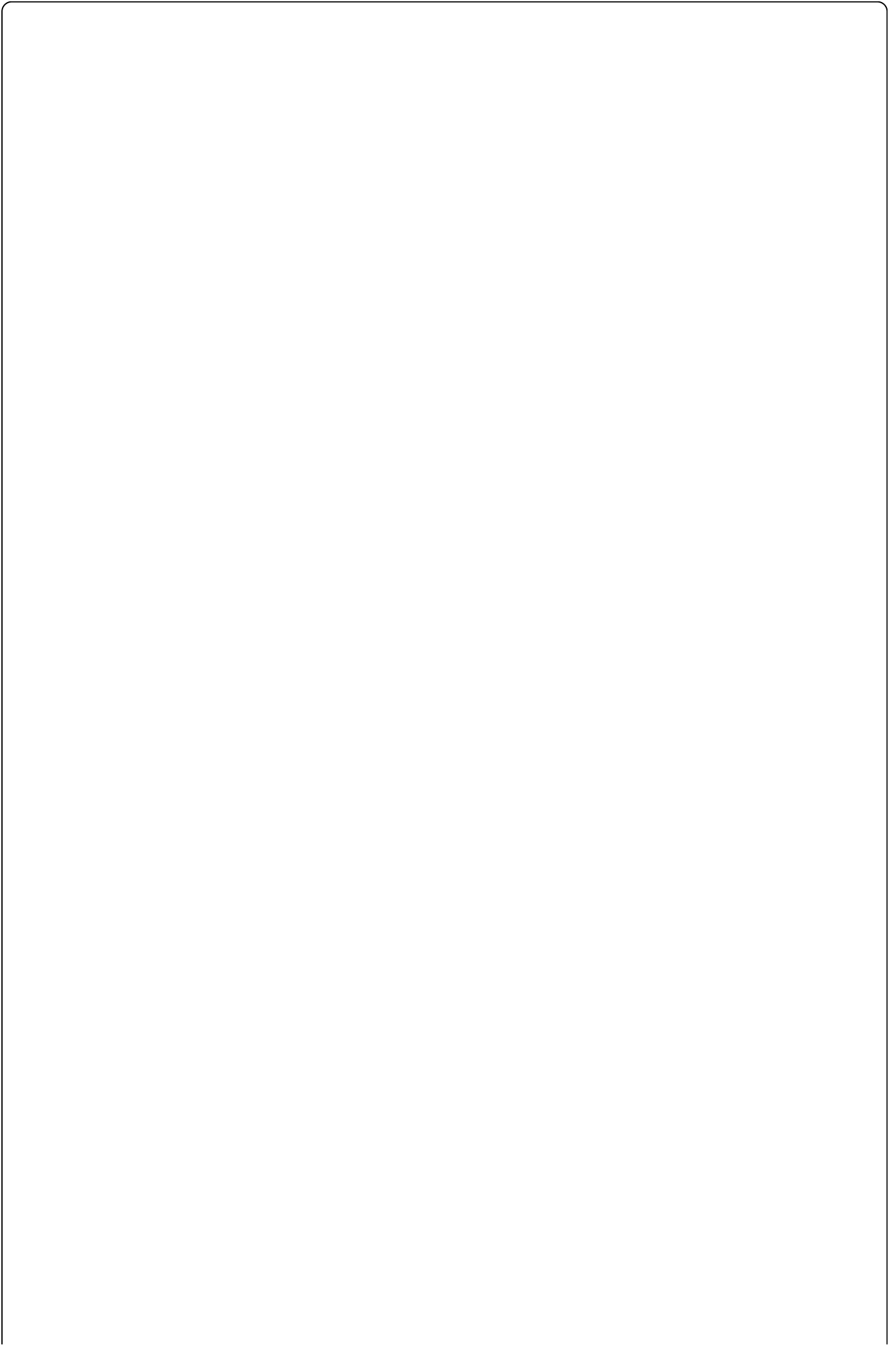
4.1 Component responsibilities

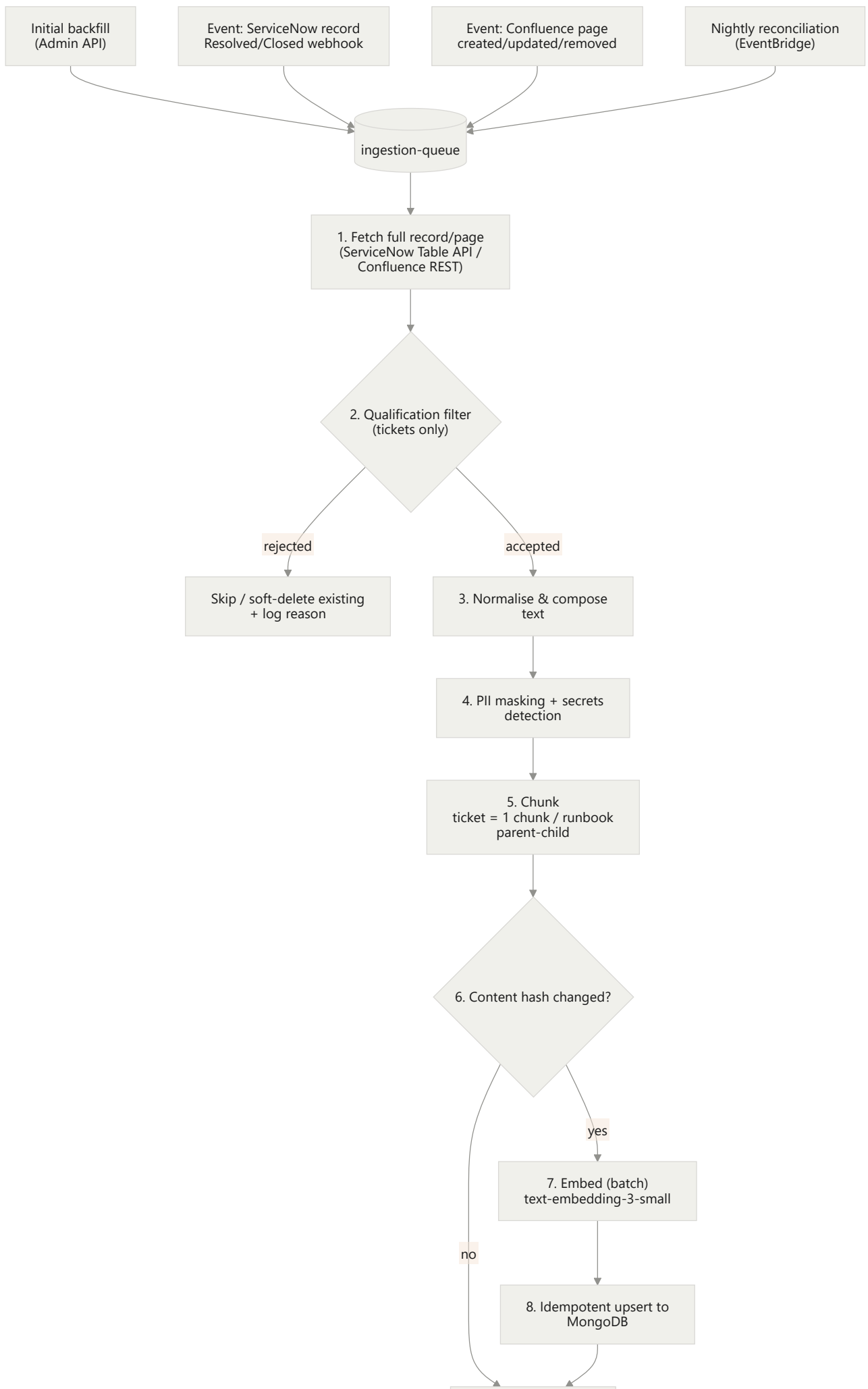
Component	Responsibility
React SPA (S3 + CloudFront)	Agent view (similar incidents, summaries, insight, refresh, feedback); Admin view (dictionary, configuration, ingestion jobs, evaluation, KPI dashboard)
api-service (Fargate)	REST API, webhook reception and HMAC verification, JWT validation, RBAC, enqueueing jobs, serving results
ingestion-worker (Fargate)	Backfill, event-driven sync, nightly reconciliation; qualification, masking, chunking, embedding, upsert
copilot-worker (Fargate)	Retrieval pipeline end to end; writes results to MongoDB and Work Notes to ServiceNow
SQS queues + DLQs	Buffer spikes (e.g. major-outage ticket bursts), retries, dead-lettering of failed jobs
EventBridge Scheduler	Triggers nightly reconciliation
MongoDB Atlas	Vector index (<code>\$vectorSearch</code>), BM25 full-text index (Atlas Search <code>\$search</code>), operational collections
Amazon Cognito	Federation with corporate IdP (SAML/OIDC), issues JWTs with group claims
Secrets Manager	ServiceNow, Confluence, OpenAI and Groq credentials; webhook HMAC secrets
CloudWatch	Logs, metrics, alarms

5. Ingestion Pipeline (Critical)

Purpose: ingest qualified ServiceNow records and Confluence runbooks into MongoDB as vector embeddings with searchable text and metadata.

Core flow: `Input → Qualify → Mask → Chunk → Embed → Store in MongoDB`





5.1 Triggers (Q8)

Trigger	Mechanism	Purpose
Initial backfill	Admin calls <code>POST /api/v1/ingestion/backfill</code> ; worker pages through ServiceNow Table API and Confluence REST API	Load the historical corpus
Event-driven (ServiceNow)	Async Business Rule on Incident / Problem / Change when state becomes <i>Resolved</i> or <i>Closed</i> → outbound REST (RESTMessageV2) with HMAC signature	New fixes searchable within seconds
Event-driven (Confluence)	Confluence webhooks for page created / updated / removed	Runbook changes reflected immediately
Nightly reconciliation	EventBridge Scheduler → <code>POST /api/v1/ingestion/reconciliation/run</code> (internal)	Catch missed events, bulk edits, deletions; retire out-of-window records

Incremental pulls use checkpoints stored in MongoDB (ServiceNow `sys_updated_on`, Confluence last-modified version).

5.2 Qualification filter (Q9)

Applied to ServiceNow records only. All thresholds are configurable via the Admin configuration API.

Rule	Default
State	Resolved or Closed
Resolution content present	Incident: <code>close_notes</code> ; Problem: workaround / fix / cause notes; Change: closure / implementation notes
Retention window	Resolved within the last 24 months
Excluded close codes	Configurable list (e.g. <i>Duplicate</i> , <i>No user response</i> , <i>Cancelled</i> , auto-closed codes — mapped to the instance's actual close code values)
Minimum resolution-note length	Configurable (default 30 characters after trimming)

Records that fail qualification are not embedded. If a previously ingested record no longer qualifies (e.g. reopened, or aged beyond the window), it is soft-deleted (`active: false`).

5.3 Text composition

Ticket document template (ServiceNow):

```
[Type] Incident | [Number] INC0012345
[Short description] ...
[Description] ...
[Configuration item] ...
[Resolution / Close notes] ...
[Close code] ...
```

Runbook document: Confluence storage format is converted to clean text/Markdown. Macros and navigation noise are removed, and heading hierarchy, numbered steps and code blocks are preserved.

5.4 PII and secrets masking (Q17)

Performed **before** embedding and storage, in a shared `masking` module that the retrieval pipeline also uses.

Category	Examples	Replacement
Person identifiers	Caller names, user IDs, employee IDs	<USER>
Contact details	Emails, phone numbers	<EMAIL>, <PHONE>
Credentials / secrets	Passwords, API keys, tokens, private keys, connection strings	<SECRET>
Kept in clear text	Hostnames, application names, error codes, service names	— (required for search)

Detection combines deterministic patterns (regex, key-format signatures, entropy checks for secrets) with ServiceNow reference fields (caller, opened_by), which are known PII and are never included in the composed text. Masking statistics are logged per document without logging the raw values.

5.5 Chunking (Q10)

Source	Strategy
ServiceNow ticket	One chunk per record (composed text). Key fields stored as metadata.
Confluence runbook	Parent-child: parent = full section split by heading; children = smaller sub-chunks of each section (configurable size, default ~250 tokens with small overlap). Children are embedded and searched; the parent section is passed to the LLM.

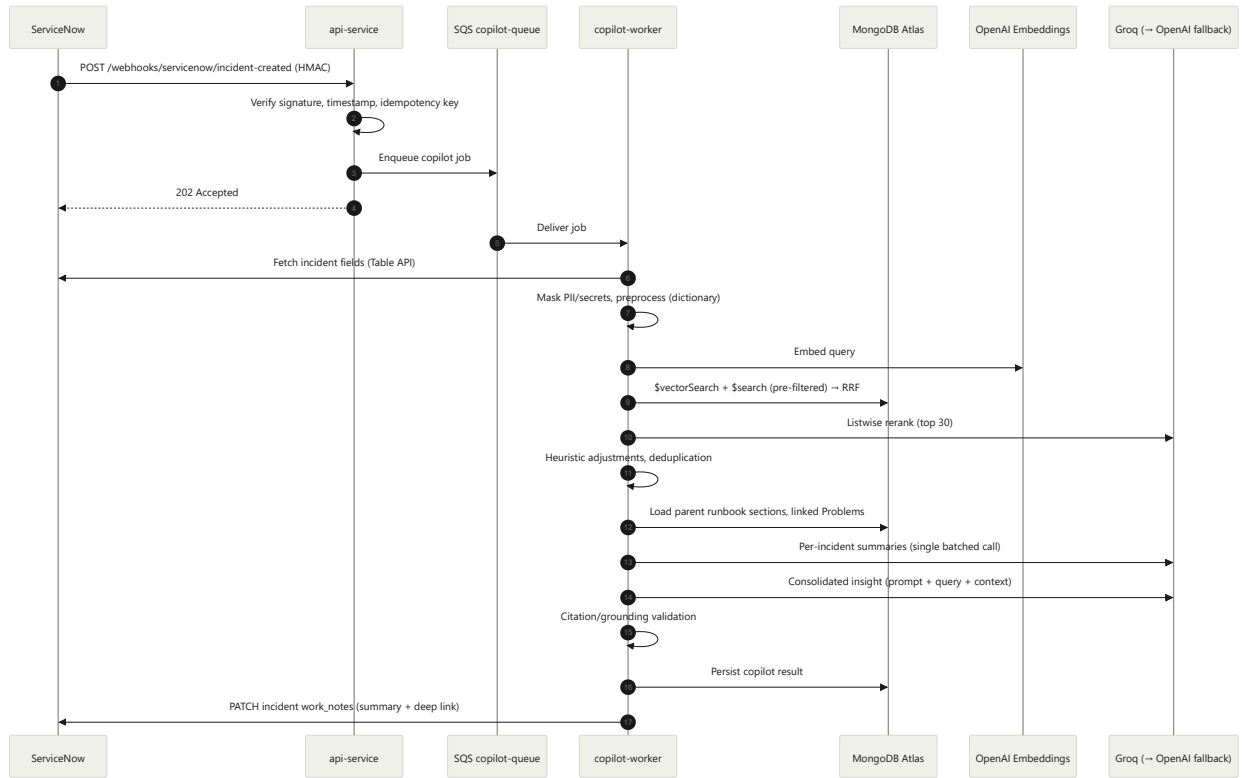
5.6 Embedding (Q11)

Item	Value
Model	OpenAI <code>text-embedding-3-small</code>
Dimensions	1536
Abstraction	<code>EmbeddingProvider</code> interface (<code>embed(texts[]): number[][]</code>) so Mistral or other OpenAI models can be benchmarked through configuration
Batching	Batched requests with retry and exponential backoff on 429/5xx
Traceability	<code>embeddingModel</code> and <code>embeddingVersion</code> stored on every chunk; a model change triggers a controlled full re-embed into a new index
Change detection	SHA-256 <code>contentHash</code> of masked text; unchanged content is not re-embedded

6. Retrieval Pipeline (Critical)

Purpose: for a new Incident, retrieve, rank and summarise the most relevant past knowledge.

Core flow: `Query → Preprocessing → Vector + BM25 → Hybrid (RRF) → Reranking → Deduplication → Summarization → Prompt + Query + Context → LLM → Output`



6.1 Step detail

#	Step	Implementation
1	Trigger	Automatic (Incident created webhook) or on-demand refresh from the React app (Q6).
2	Query construction	Short description + description + configuration item name + any agent work notes added (for refresh).
3	Masking	Same masking module as ingestion (Q17).
4	Normalisation	Lower-casing (except error codes), punctuation and whitespace cleanup, spelling variants of common terms per dictionary.
5	Abbreviation expansion	Dictionary lookup (e.g. <code>pwd</code> → <code>password</code>), <code>lappy</code> → <code>laptop</code>).
6	Synonym expansion	Dictionary lookup (e.g. <code>VPN</code> ↔ <code>remote access</code> ↔ <code>GlobalProtect</code>). Expanded synonyms are added to the BM25 query; the vector query uses normalised + abbreviation-expanded text to avoid diluting the embedding.
7	Vector search	<code>\$vectorSearch</code> on <code>knowledge_chunks.embedding</code> , pre-filtered on <code>active</code> , <code>sourceType</code> , <code>resolvedAt</code> window. Default <code>limit</code> 50.
8	BM25 search	Atlas Search <code>\$search</code> (compound query) on <code>shortDescription</code> , <code>content</code> , <code>resolutionText</code> , with an analyzer that preserves error codes and hostnames as tokens; same pre-filters. Default top 50.
9	Hybrid fusion	Reciprocal Rank Fusion ($\text{score} = \sum 1 / (k + \text{rank})$), default <code>k = 60</code>). Implemented with MongoDB's native <code>\$rankFusion</code> stage where the cluster version supports it, otherwise application-side RRF over the two parallel queries.
10	LLM reranking	Groq listwise call over the top 30 fused candidates, returning a relevance score (0-1) per candidate in JSON.
11	Heuristic adjustment	$\text{final} = w1 \cdot \text{llmScore} + w2 \cdot \text{recency} + w3 \cdot \text{noteQuality} + w4 \cdot \text{sameCI} + w5 \cdot \text{sameAssignmentGroup} - w6 \cdot \text{reopenPenalty}$. Same-CI and same-group are soft boosts (Q13). Weights configurable.
12	Deduplication	See §6.2.
13	Top-N selection	Default N = 5 representatives, configurable.
14	Summarization	One batched LLM call returns, for each representative: <i>issue</i> , <i>root cause</i> , <i>resolution</i> (2-3 lines). This compresses context for the next step.

#	Step	Implementation
15	Prompt + Query + Context → LLM	Consolidated insight generated from the masked query, per-incident summaries, linked Problem records and parent runbook sections, with mandatory citations.
16	Grounding validation	Every citation must reference an ID present in the supplied context; uncited or invalid claims are removed and the result is flagged <code>groundingWarnings</code> .
17	Persist & deliver	Result stored in <code>copilot_results</code> ; concise Work Note posted to ServiceNow; full result served to React.

6.2 Deduplication (Q15)

Applied in order:

1. **Exact ID** — remove repeated entries of the same record or chunk.
2. **Parent-child collapse** — multiple matched runbook child chunks collapse into one parent section.
3. **Near-duplicate clustering** — candidates with embedding cosine similarity ≥ 0.95 (configurable) are clustered; the highest-scoring member is the representative, labelled "+N similar incidents".
4. **ServiceNow relationship grouping** — incidents sharing a `parent_incident` or linked to the same Problem (`problem_id`) are grouped; the **Problem record** becomes the representative when it has been ingested, since it holds the confirmed root cause and workaround.

Group members stay available (IDs and links) in the React detail view.

6.3 LLM provider strategy (Q16)

Aspect	Design
Primary	Groq (configurable model ID) for reranking, per-incident summaries and consolidated insight
Fallback	OpenAI (configurable model ID), invoked automatically on Groq 5xx, 429, or timeout
Resilience	Per-call timeout, retry with backoff, circuit breaker that routes directly to OpenAI while Groq is unhealthy
Interface	<code>LlmProvider</code> abstraction (both providers expose OpenAI-compatible chat APIs); a single prompt set for both
Output format	Structured JSON validated against a schema (e.g. Zod); one repair retry on invalid JSON
Metering	Tokens, latency, provider and failover events recorded per call in <code>llm_usage</code>

6.4 Prompt design (templates)

Reranking prompt (system, abridged):

```
You are an IT service desk retrieval judge. Given a NEW TICKET and a list of
CANDIDATES
(past incidents, problems, changes, runbook sections), score how likely each
candidate
describes the same underlying issue or its fix. Return JSON only:
[{"id": "<candidateId>", "score": <0.0-1.0>}]. Do not invent IDs.
```

Per-incident summary prompt (system, abridged):

```
For each record provided, summarise in at most 3 short lines:
issue, root_cause (or "not recorded"), resolution. Use only the record's
text.
Return JSON: [{"id": "...", "issue": "...", "root_cause": "...",
"resolution": "..."}].
```

Consolidated insight prompt (system, abridged):

You assist a service desk agent. Using ONLY the context provided (similar records and runbook sections), write a concise insight: the most likely cause, the most common successful resolution, and the relevant runbook procedure. Cite every claim with the source IDs in square brackets, e.g. [PRB0040012], [RB:VPN-Gateway#restart]. If the context is insufficient, say so. Never include credentials or personal data.

Return JSON: {"insight": "...", "citations": ["..."], "confidence": "high|medium|low"}.

6.5 ServiceNow Work Note format (Q7)

[Resolution Copilot] 5 similar records found (top match: PRB0040012, relevance 0.91)
Insight: 4 of 5 similar incidents were resolved by renewing the VPN gateway certificate
(see runbook "VPN Gateway - Certificate Renewal"). [PRB0040012][INC0098811]
Top matches: PRB0040012, INC0098811, INC0097302, CHG0031190, INC0090457
Full details & feedback: <https://<copilot-domain>/tickets/INC0012345>

On refresh, a new Work Note is posted only if the top results changed materially (configurable), which avoids cluttering the ticket.

7. Data Model (MongoDB Atlas)

7.1 Collections

Collection	Purpose	Key fields
knowledge_chunks	Searchable units (tickets and runbook child chunks)	<code>_id</code> , <code>sourceType</code> (<code>incident</code> <code>problem</code> <code>change</code> <code>runbook</code>), <code>sourceId</code> , <code>number</code> , <code>chunkType</code> , <code>parentId</code> , <code>shortDescription</code> , <code>content</code> (masked), <code>resolutionText</code> , <code>embedding</code> [1536], <code>embeddingModel</code> , <code>embeddingVersion</code> , <code>contentHash</code> , <code>metadata</code> { <code>cmdbCi</code> , <code>assignmentGroup</code> , <code>closeCode</code> , <code>problemId</code> , <code>parentIncident</code> , <code>reopenCount</code> , <code>resolvedAt</code> , <code>noteQuality</code> , <code>url</code> }, <code>active</code> , <code>ingestedAt</code> , <code>updatedAt</code>
runbook_parents	Full runbook sections passed to the LLM	<code>_id</code> , <code>pageId</code> , <code>title</code> , <code>sectionPath</code> , <code>content</code> (masked), <code>url</code> , <code>pageVersion</code> , <code>active</code>
sync_checkpoints	Incremental sync state	<code>source</code> , <code>table</code> , <code>lastUpdatedOn</code> , <code>lastRunAt</code>
ingestion_jobs	Backfill / event / reconciliation job tracking	<code>jobId</code> , <code>type</code> , <code>source</code> , <code>status</code> , <code>counts</code> { <code>fetched</code> , <code>qualified</code> , <code>rejected</code> , <code>embedded</code> , <code>skippedUnchanged</code> , <code>failed</code> }, <code>startedAt</code> , <code>endedAt</code> , <code>errors[]</code>
dictionary_entries	Query preprocessing dictionary	<code>_id</code> , <code>type</code> (<code>normalization</code> <code>abbreviation</code> <code>synonym</code>), <code>term</code> , <code>expansions[]</code> , <code>active</code> , <code>updatedBy</code> , <code>updatedAt</code>
copilot_results	Generated results per ticket run	<code>resultId</code> , <code>ticketNumber</code> , <code>trigger</code> (<code>auto</code> <code>refresh</code>), <code>query</code> (masked), <code>items[]</code> { <code>id</code> , <code>type</code> , <code>score</code> , <code>summary</code> , <code>groupMembers[]</code> , <code>url</code> }, <code>insight</code> , <code>citations[]</code> , <code>confidence</code> , <code>groundingWarnings[]</code> , <code>provider</code> , <code>latencyMs</code> , <code>createdAt</code>
copilot_jobs	Async job status	<code>jobId</code> , <code>ticketNumber</code> , <code>status</code> , <code>resultId</code> , <code>error</code>
feedback	Agent feedback	<code>resultId</code> , <code>ticketNumber</code> , <code>itemId</code> (or <code>insight</code>), <code>rating</code> (<code>helpful</code> <code>not_helpful</code>), <code>resolvedBy</code> (bool), <code>comment</code> , <code>userId</code> , <code>createdAt</code>

Collection	Purpose	Key fields
<code>golden_set</code>	Evaluation dataset	<code>caseId</code> , <code>ticketText</code> , <code>expectedIds[]</code> , <code>notes</code>
<code>evaluation_runs</code>	Evaluation results	<code>runId</code> , <code>configSnapshot</code> , <code>metrics</code> { <code>precisionAtK</code> , <code>recallAtK</code> , <code>mrr</code> , <code>groundedness</code> }, <code>perCase[]</code> , <code>status</code> , <code>createdAt</code>
<code>llm_usage</code>	Cost and latency metering	<code>callType</code> , <code>provider</code> , <code>model</code> , <code>inputTokens</code> , <code>outputTokens</code> , <code>latencyMs</code> , <code>failover</code> , <code>ticketNumber</code> , <code>createdAt</code>
<code>app_config</code>	Versioned runtime configuration	qualification rules, thresholds, weights, top-K/N, model IDs, prompts version
<code>audit_log</code>	Admin and security audit trail	<code>actor</code> , <code>action</code> , <code>target</code> , <code>before</code> , <code>after</code> , <code>timestamp</code>

7.2 Indexes

Index	Type	Definition
<code>kc_vector_idx</code>	Atlas Vector Search	<code>embedding</code> (1536, cosine); filter fields: <code>sourceType</code> , <code>active</code> , <code>metadata.resolvedAt</code>
<code>kc_text_idx</code>	Atlas Search (BM25)	<code>shortDescription</code> , <code>content</code> , <code>resolutionText</code> with custom analyzer preserving error codes / hostnames; filter fields as above
<code>kc_source_unique</code>	B-tree, unique	{ <code>sourceType</code> , <code>sourceId</code> , <code>chunkId</code> }
<code>results_ticket</code>	B-tree	{ <code>ticketNumber</code> , <code>createdAt</code> : -1}
<code>feedback_result</code>	B-tree	{ <code>resultId</code> , <code>itemId</code> }
<code>dict_term_unique</code>	B-tree, unique	{ <code>type</code> , <code>term</code> }

8. API Specification

Base URL: `https://<copilot-domain>/api/v1` **Auth:** Bearer JWT issued by Amazon Cognito (users); HMAC signature (webhooks). **Conventions:** JSON; ISO-8601 timestamps; long-running operations return `202 Accepted` with a `jobId`; errors follow `{ "error": { "code", "message", "correlationId" } }`.

8.1 Webhooks — event intake (ServiceNow / Confluence → Copilot)

Method	Endpoint	Functionality	Caller / Auth
POST	<code>/webhooks/servicenow/incident-created</code>	Triggers the retrieval pipeline for a new Incident; enqueues a copilot job	ServiceNow Business Rule / HMAC
POST	<code>/webhooks/servicenow/record-resolved</code>	Event-driven ingestion of a Resolved/Closed Incident, Problem or Change	ServiceNow Business Rule / HMAC
POST	<code>/webhooks/confluence/page-event</code>	Event-driven ingestion of runbook page created / updated / removed	Confluence webhook / HMAC

Webhook request headers: `X-Signature` (HMAC-SHA256 of timestamp + body), `X-Timestamp` (replay window, default 5 minutes), `X-Idempotency-Key`.

Example — `POST /webhooks/servicenow/incident-created`

```
json
{ "sysId": "a1b2c3...", "number": "INC0012345", "table": "incident", "event": "
```

Response: `202 Accepted` → `{ "jobId": "cj_01H..." }`

8.2 Retrieval / Copilot (Agent functionality)

Method	Endpoint	Functionality	Role
GET	<code>/tickets/{ticketNumber}/similar-incidents</code>	Latest Copilot result for a ticket (ranked items, per-incident summaries, consolidated insight, citations)	Agent, Admin
GET	<code>/tickets/{ticketNumber}/similar-incidents/history</code>	All Copilot runs for the ticket (auto + refresh)	Agent, Admin
POST	<code>/tickets/{ticketNumber}/similar-incidents/refresh</code>	On-demand refresh; re-runs the retrieval pipeline with latest ticket content	Agent, Admin
GET	<code>/copilot/jobs/{jobId}</code>	Status of an automatic or refresh job (<code>queued</code>) (<code>running</code>) (<code>completed</code>) (<code>failed</code>)	Agent, Admin
GET	<code>/copilot/results/{resultId}</code>	Retrieve one specific result, including group members	Agent, Admin

Example — `GET /tickets/INC0012345/similar-incidents`

json

```

{
  "resultId": "cr_01H...",
  "ticketNumber": "INC0012345",
  "generatedAt": "2026-10-01T09:15:22Z",
  "items": [
    {
      "id": "PRB0040012",
      "type": "problem",
      "score": 0.91,
      "summary": {
        "issue": "VPN connections failing with error 809 for remote users",
        "rootCause": "Expired certificate on VPN gateway",
        "resolution": "Certificate renewed and gateway service restarted"
      },
      "groupMembers": ["INC0098811", "INC0097302"],
      "url": "https://<instance>.service-now.com/problem.do?sys_id=..."
    }
  ],
  "insight": {
    "text": "Most likely cause is an expired gateway certificate ... [PRB0040012]",
    "citations": ["PRB0040012", "RB:VPN-Gateway#cert-renewal"],
    "confidence": "high"
  },
  "groundingWarnings": []
}

```

8.3 Feedback (Agent functionality)

Method	Endpoint	Functionality	Role
POST	<code>/copilot/results/{resultId}/feedback</code>	Rate an item or the insight (helpful / not helpful, optional comment)	Agent, Admin
POST	<code>/copilot/results/{resultId}/resolved-by</code>	Mark which suggested record resolved the ticket	Agent, Admin
GET	<code>/admin/feedback</code>	List and filter feedback (date, rating, assignment group)	Admin

Example — `POST /copilot/results/cr_01H.../feedback`

```
json
```

```
{ "target": "item", "itemId": "PRB0040012", "rating": "helpful", "comment": "Ex
```

8.4 Ingestion management (Admin functionality)

Method	Endpoint	Functionality	Role
POST	<code>/ingestion/backfill</code>	Start initial or re-backfill for a source (<code>servicenow</code>) table(s) or (<code>confluence</code>) space(s)), optional date range	Admin
POST	<code>/ingestion/reconciliation/run</code>	Run reconciliation on demand (also invoked nightly by EventBridge)	Admin / Scheduler
GET	<code>/ingestion/jobs</code>	List ingestion jobs with status and counts	Admin
GET	<code>/ingestion/jobs/{jobId}</code>	Job detail (fetched, qualified, rejected with reasons, embedded, failed)	Admin
POST	<code>/ingestion/jobs/{jobId}/cancel</code>	Cancel a running job	Admin
POST	<code>/ingestion/dlq/redrive</code>	Re-process messages from the ingestion dead-letter queue	Admin
GET	<code>/ingestion/sources/status</code>	Per-source health: last sync checkpoint, document counts, last error	Admin
POST	<code>/ingestion/re-embed</code>	Controlled full re-embed after an embedding model change	Admin

Method	Endpoint	Functionality	Role
DELETE	<code>/ingestion/documents/{sourceType}/{sourceId}</code>	Purge a record and its chunks from the index (e.g. data removal request)	Admin

8.5 Query preprocessing dictionary (Admin functionality)

Method	Endpoint	Functionality	Role
GET	<code>/admin/dictionary/entries</code>	List entries (filter by type, term, active)	Admin
POST	<code>/admin/dictionary/entries</code>	Create entry (normalization / abbreviation / synonym)	Admin
PUT	<code>/admin/dictionary/entries/{entryId}</code>	Update entry	Admin
DELETE	<code>/admin/dictionary/entries/{entryId}</code>	Delete entry	Admin
POST	<code>/admin/dictionary/import</code>	Bulk import (CSV/JSON) with validation report	Admin
GET	<code>/admin/dictionary/export</code>	Export the dictionary	Admin
POST	<code>/admin/dictionary/preview</code>	Show normalised, expanded BM25 and vector query text for sample input	Admin

8.6 Configuration (Admin functionality)

Method	Endpoint	Functionality	Role
GET	<code>/admin/config</code>	Current configuration: qualification rules, retention window, close-code exclusions, top-K/N, RRF(k), rerank weights, near-duplicate threshold, model IDs, prompt version	Admin
PUT	<code>/admin/config</code>	Update configuration (versioned, audited)	Admin
GET	<code>/admin/config/history</code>	Configuration versions and change history	Admin

8.7 Evaluation (Admin functionality)

Method	Endpoint	Functionality	Role
GET	<code>/admin/evaluation/golden-set</code>	List golden-set cases	Admin
POST	<code>/admin/evaluation/golden-set</code>	Add a case (ticket text + expected record IDs)	Admin
PUT	<code>/admin/evaluation/golden-set/{caseId}</code>	Update a case	Admin
DELETE	<code>/admin/evaluation/golden-set/{caseId}</code>	Remove a case	Admin
POST	<code>/admin/evaluation/runs</code>	Run evaluation against current or candidate configuration	Admin
GET	<code>/admin/evaluation/runs</code>	List runs with headline metrics	Admin
GET	<code>/admin/evaluation/runs/{runId}</code>	Run detail: Precision@K, Recall@K, MRR, groundedness, per-case results	Admin

8.8 KPI & cost dashboard (Admin functionality)

Method	Endpoint	Functionality	Role
GET	<code>/admin/metrics/kpis?from&to&assignmentGroup</code>	MTTR comparison, adoption rate, helpfulness rate, resolved-by rate	Admin
GET	<code>/admin/metrics/latency?from&to</code>	Pipeline and per-stage latency (p50/p95/p99)	Admin
GET	<code>/admin/metrics/costs?from&to</code>	Embedding and LLM token cost per ticket, per provider, failover counts	Admin
GET	<code>/admin/metrics/ingestion?from&to</code>	Ingestion volumes, rejection reasons, DLQ counts	Admin

8.9 Identity & platform

Method	Endpoint	Functionality	Role
GET	<code>/auth/me</code>	Current user profile and roles (from Cognito JWT claims)	Agent, Admin
GET	<code>/health/live</code>	Liveness probe	Load balancer
GET	<code>/health/ready</code>	Readiness (MongoDB, SQS, Secrets Manager reachable)	Load balancer

Sign-in, sign-out and token refresh are handled by the Cognito Hosted UI / OIDC flow in the React app, so no custom login endpoints are exposed.

9. Security Architecture (Q17, Q18)

9.1 Identity and access

Actor	Mechanism
Agents / Admins	Corporate IdP (SAML / OIDC) federated through Amazon Cognito; MFA enforced at the IdP; JWT validated by api-service middleware
Roles	IdP groups mapped to Agent and Admin application roles
ServiceNow → Copilot	HMAC-SHA256 signed webhooks, timestamp replay protection, idempotency keys, source IP allow-list (AWS WAF)
Confluence → Copilot	Webhook secret signature verification + IP allow-list
Copilot → ServiceNow	Dedicated integration user with least-privilege roles (read on incident / problem / change; write limited to <code>work_notes</code>), OAuth 2.0 credentials
Copilot → Confluence	Service account API token, read-only on configured spaces

9.2 RBAC matrix

Capability	Agent	Admin
View similar incidents / insight	✓	✓
Refresh results	✓	✓
Submit feedback / resolved-by	✓	✓
Manage dictionary	✗	✓
Manage configuration	✗	✓
Trigger ingestion / re-embed / purge	✗	✓
Evaluation and KPI dashboard	✗	✓

9.3 Data protection

- PII masked and secrets removed **before** embedding, storage and every LLM call (ingestion and query time).
- TLS 1.2+ in transit; MongoDB Atlas encryption at rest; S3 server-side encryption.
- Private network path from Fargate to MongoDB Atlas via AWS PrivateLink; outbound provider traffic via NAT Gateway.
- All credentials in AWS Secrets Manager with rotation; no secrets in images or environment files.
- Prompt-injection hardening: retrieved content is inserted as delimited data, system prompts instruct the model to ignore instructions within context, and output is schema-validated.
- Audit log for all Admin actions and configuration changes.

10. AWS Deployment Architecture (Q19)

Layer	Service	Notes
Frontend	S3 + CloudFront + AWS WAF	React SPA; WAF rules, rate limiting
Ingress	Application Load Balancer (HTTPS)	Routes <code>/api/*</code> and <code>/webhooks/*</code> to api-service
Compute	Amazon ECS on Fargate	<code>api-service</code> , <code>ingestion-worker</code> , <code>copilot-worker</code> as separate services; images in Amazon ECR
Messaging	Amazon SQS (standard) + DLQs	<code>ingestion-queue</code> , <code>copilot-queue</code> ; visibility timeouts sized per job type
Scheduling	Amazon EventBridge Scheduler	Nightly reconciliation
Database	MongoDB Atlas (same AWS region)	Vector Search + Atlas Search; PrivateLink
Identity	Amazon Cognito	Federated with corporate IdP
Secrets	AWS Secrets Manager	Provider keys, ServiceNow/Confluence credentials, HMAC secrets
Observability	Amazon CloudWatch (logs, metrics, alarms)	Structured JSON logs with correlation IDs
Network	VPC with public (ALB) and private (Fargate) subnets across ≥ 2 AZs; NAT Gateway	Security groups restrict inbound to the ALB

10.1 Scaling

Service	Scaling signal
api-service	CPU / request count per target
copilot-worker	SQS <code>copilot-queue</code> depth and age of oldest message
ingestion-worker	SQS <code>ingestion-queue</code> depth; temporarily scaled up for backfill

10.2 CI/CD (recommended baseline)

Build → unit tests → container image to ECR → **golden-set evaluation gate** (Q20) for changes to prompts, models, chunking or retrieval configuration → deploy to ECS (rolling)

→ smoke test via `/health/ready`.

11. Quality, Feedback & Observability (Q20)

11.1 Operational monitoring (CloudWatch)

Metric	Alarm condition (initial, tunable)
API 5xx rate	Above threshold for 5 minutes
Copilot end-to-end latency (webhook → Work Note)	p95 above target
SQS age of oldest message	Rising beyond threshold
DLQ message count	> 0
LLM failover rate (Groq → OpenAI)	Sustained increase
Webhook signature failures	Spike (possible attack or misconfiguration)
Ingestion job failures	Any failed job

11.2 Agent feedback

- Helpful / not-helpful rating per similar incident and for the consolidated insight.
- *"This resolved my ticket"* marker per item.
- Optional free-text comment.

11.3 Offline evaluation (golden set)

Metric	Measures
Precision@K	Share of returned items that are correct matches
Recall@K	Share of expected matches found in top K
MRR	How high the first correct match ranks
Groundedness	Share of insight claims with valid citations to supplied context

Evaluation runs are mandatory before releasing changes to the embedding model, LLM model IDs, prompts, chunking parameters, dictionary bulk changes or ranking weights.

11.4 KPI & cost dashboard

KPI	Definition
MTTR comparison	MTTR of tickets where Copilot results were viewed vs. not viewed, and trend against the pre-launch baseline
Adoption rate	% of Copilot-processed tickets where an agent opened the detail view
Helpfulness rate	Helpful ratings ÷ total ratings
Resolved-by rate	% of tickets where a suggested record was marked as the resolution
Latency	p50 / p95 per pipeline stage
Cost per ticket	Embedding + LLM tokens × unit price, by provider

12. Non-Functional Requirements (initial targets, to be validated in pilot)

Attribute	Target
Timeliness	Result available (Work Note posted) within ~15 seconds of Incident creation at p95
Freshness	Newly resolved records searchable within ~1 minute of the resolution event
Availability	99.5% for api-service during business hours; queued jobs survive worker failures
Resilience	Automatic LLM failover; SQS retries with DLQ; idempotent processing
Scalability	Absorb major-outage bursts via queueing and worker autoscaling
Security	No raw PII or secrets in vector store, logs or LLM prompts
Auditability	All Work Notes, results, feedback and admin changes persisted

13. Backend Module Structure (Node.js + TypeScript)

```
src/
├─ api/                # Express/Fastify routes, controllers, request
validation
|   ├─ webhooks/       # HMAC verification, idempotency
|   ├─ copilot/        # results, refresh, jobs
|   ├─ feedback/
|   └─ admin/          # dictionary, config, ingestion, evaluation, metrics
├─ auth/               # Cognito JWT validation, RBAC guards
├─ connectors/
|   ├─ servicenow/     # Table API client, work-note writer
|   └─ confluence/     # REST client, storage-format → text converter
├─ ingestion/          # qualification, composition, chunking, upsert,
reconciliation
├─ retrieval/
|   ├─ preprocessing/ # normalization, abbreviation & synonym expansion
|   ├─ search/         # $vectorSearch, $search, RRF
|   ├─ rerank/         # LLM rerank + heuristics
|   └─ dedup/          # exact, parent-child, clustering, relationship
grouping
|   └─ generation/     # summaries, insight, grounding validation
├─ providers/
|   ├─ embedding/      # EmbeddingProvider (OpenAI; Mistral-ready)
|   └─ llm/            # LlmProvider (Groq primary, OpenAI fallback,
circuit breaker)
├─ security/masking/   # PII & secrets detection (shared)
├─ workers/            # SQS consumers: ingestion-worker, copilot-worker
├─ evaluation/         # golden-set runner and metrics
├─ metrics/            # KPI and cost aggregation
└─ shared/             # config, logging, errors, db client
```

14. Risks and Mitigations

Risk	Mitigation
Poor historical resolution notes reduce match quality	Quality filter (Q9); feedback data used to tune thresholds; Problem-record grouping surfaces authoritative fixes
LLM hallucination in summaries / insight	Context-only prompts, mandatory citations, grounding validation, golden-set groundedness metric
Groq outage or rate limits	Automatic OpenAI failover with circuit breaker
PII or secrets leakage	Masking at ingestion and query time; secrets detection; no raw text in logs
Webhook spoofing / cost abuse	HMAC signatures, replay window, IP allow-list, WAF rate limits
Missed events → stale index	Nightly reconciliation; DLQ monitoring and redrive
Embedding model change requires re-embed	Model/version stored per chunk; controlled re-embed endpoint; evaluation gate
Dictionary drift or errors	Admin preview endpoint, audit log, evaluation run after bulk changes
Work Note clutter on refresh	Post only on material change

15. Future Enhancements (not in current scope)

These items were considered in the questionnaire but deliberately excluded. They can be added without changing the core architecture.

- Ingest ServiceNow Knowledge Base articles (V).
- Ingest Teams/Slack incident threads and post-mortems (V), with enhanced PII handling.
- Live CMDB and monitoring alert enrichment (No Vector).
- Auto-triage (category, priority, assignment group).
- Draft customer replies for agent review.
- React panel embedded inside the ServiceNow Incident form / Workspace.
- Mining abbreviation and synonym candidates from tickets for admin approval.
- Task-based LLM routing (e.g. a stronger model for the consolidated insight).

End of document.